Experiment-1: Creating NumPy Arrays

Perform the following operations using NumPy:

i) Create a Basic ndarray

ii) Create an Array of Zeros

iii) Create an Array of Ones

iv) Generate Random Numbers in ndarray

v) Create an Array of your Choice

vi) Create an Identity Matrix in NumPy

vii) Create an Evenly Spaced ndarray

AIM

To understand the creation of NumPy arrays using different built-in functions and generate arrays of various types for numerical computations.

Description

NumPy (Numerical Python) is an open-source Python library used for scientific computing and numerical analysis. It provides support for multidimensional arrays and numerous mathematical functions to perform fast operations on these arrays.

Unlike Python lists, NumPy arrays occupy contiguous memory locations, making computations significantly faster and more memory efficient.

The fundamental object in NumPy is the **ndarray (N-dimensional array)**.

NumPy provides several methods to create arrays according to different requirements.

1. Basic ndarray

The **array()** function converts Python lists or tuples into NumPy arrays.

Syntax

```
numpy.array(object)
```

Example

```
import numpy as np

arr = np.array([10,20,30,40])
print(arr)
```

Output

```
[10 20 30 40]
```

2. Array of Zeros

The **zeros()** function creates an array where every element is zero.

Syntax

```
numpy.zeros(shape)
```

Example

```
import numpy as np

arr=np.zeros((3,4))
print(arr)
```

Output

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

3. Array of Ones

The **ones()** function creates an array filled entirely with ones.

Syntax

```
numpy.ones(shape)
```

Example

```
import numpy as np

arr=np.ones((2,5))
print(arr)
```

Output

```
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
```

4. Random Numbers in ndarray

Random arrays can be generated using the **random** module.

Common functions include

- random.rand()
- random.randint()
- random.random()

Example

```
import numpy as np

arr=np.random.randint(1,100,(3,3))
print(arr)
```

Sample Output

```
[[45 81 12]
 [34 56 92]
 [67 18 40]]
```

(Output changes every execution.)

5. Array of Your Choice

Users can create arrays using any desired values.

Example

```
import numpy as np

arr=np.array([25,36,49,64,81])
print(arr)
```

Output

```
[25 36 49 64 81]
```

6. Identity Matrix

The **identity()** function creates a square matrix with diagonal elements equal to one.

Syntax

```
numpy.identity(n)
```

Example

```
import numpy as np

arr=np.identity(4)

print(arr)
```

Output

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

7. Evenly Spaced ndarray

The **linspace()** function creates equally spaced values between two numbers.

Syntax

```
numpy.linspace(start,stop,number_of_values)
```

Example

```
import numpy as np

arr=np.linspace(0,10,6)

print(arr)
```

Output

```
[ 0. 2. 4. 6. 8. 10.]
```

Procedure

Step 1: Install Python.

Step 2: Install NumPy library.

pip install numpy

Step 3: Open Jupyter Notebook or VS Code.

Step 4: Import NumPy.

```
import numpy as np
```

Step 5: Execute each program.

Step 6: Observe the outputs.

Program

```
import numpy as np

print("Basic ndarray")
a=np.array([10,20,30,40])
print(a)

print("\nArray of Zeros")
b=np.zeros((3,3))
print(b)

print("\nArray of Ones")
c=np.ones((2,4))
print(c)

print("\nRandom Array")
d=np.random.randint(1,100,(3,3))
print(d)

print("\nArray of Choice")
e=np.array([12,24,36,48,60])
print(e)
```

```
print("\nIdentity Matrix")
f=np.identity(5)
print(f)

print("\nEvenly Spaced Array")
g=np.linspace(0,20,11)
print(g)
```

Result

Thus, different methods of creating NumPy arrays such as basic arrays, zeros, ones, random arrays, identity matrices and evenly spaced arrays were successfully implemented.

Experiment-2: The Shape and Reshaping of NumPy Array

Perform the following operations using NumPy:

i) Dimensions of NumPy Array

ii) Shape of NumPy Array

iii) Size of NumPy Array

iv) Reshaping a NumPy Array

v) Flattening a NumPy Array

vi) Transpose of a NumPy Array

AIM

To understand the properties of NumPy arrays and perform reshaping, flattening, and transpose operations on multidimensional arrays.

Description

NumPy provides several built-in attributes and functions to obtain information about an array and modify its structure without changing the data. These operations are useful in data analysis, machine learning, and scientific computing where data needs to be organized into different dimensions.

(i) Dimensions of NumPy Array

The **ndim** attribute returns the number of dimensions of an array.

- 1-D array has one dimension.
- 2-D array has two dimensions.
- 3-D array has three dimensions.

Syntax

```
array.ndim
```

Example

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60]])

print(arr.ndim)
```

Output

2

(ii) Shape of NumPy Array

The **shape** attribute returns the number of rows and columns of an array.

For a 3-D array, it returns the number of blocks, rows, and columns.

Syntax

```
array.shape
```

Example

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60]])

print(arr.shape)
```

Output

(2, 3)

(iii) Size of NumPy Array

The **size** attribute returns the total number of elements present in an array.

Syntax

```
array.size
```

Example

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60]])

print(arr.size)
```

Output

6

(iv) Reshaping a NumPy Array

The **reshape()** function changes the shape of an array without changing its data.

The total number of elements before and after reshaping must be the same.

Syntax

```
array.reshape(rows,columns)
```

Example

```
import numpy as np

arr=np.array([1,2,3,4,5,6])

new_arr=arr.reshape(2,3)

print(new_arr)
```

Output

[[1 2 3]

[4 5 6]]

(v) Flattening a NumPy Array

The **flatten()** function converts a multidimensional array into a one-dimensional array.

Syntax

```
array.flatten()
```

Example

```
import numpy as np

arr=np.array([[1,2,3],
              [4,5,6]])

print(arr.flatten())
```

Output

[1 2 3 4 5 6]

(vi) Transpose of a NumPy Array

The **transpose()** function exchanges rows and columns of a matrix.

It can also be accessed using the **T** attribute.

Syntax

```
array.transpose()
```

or

```
array.T
```

Example

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60]])

print(arr.transpose())
```

Output

[[10 40]
 [20 50]
 [30 60]]

Procedure

Step 1: Import the NumPy library.

import numpy as np

Step 2: Create a NumPy array.

Step 3: Display the dimensions using **ndim**.

Step 4: Display the shape using **shape**.

Step 5: Display the total number of elements using **size**.

Step 6: Reshape the array using **reshape()**.

Step 7: Flatten the array using **flatten()**.

Step 8: Find the transpose using **transpose()**.

Step 9: Execute the program and observe the outputs.

Program

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60]])

print("Original Array")
print(arr)

print("\nDimensions")
print(arr.ndim)

print("\nShape")
print(arr.shape)

print("\nSize")
print(arr.size)

print("\nReshaped Array")
a=np.arange(1,13)
print(a.reshape(3,4))

print("\nFlattened Array")
print(arr.flatten())

print("\nTranspose")
print(arr.transpose())
```

Output

```
Original Array
[[10 20 30]
 [40 50 60]]

Dimensions
2

Shape
(2, 3)

Size
6

Reshaped Array
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Flattened Array
[10 20 30 40 50 60]

Transpose
[[10 40]
 [20 50]
 [30 60]]
```

Result

Thus, the dimensions, shape, size, reshaping, flattening, and transpose operations on NumPy arrays were successfully implemented and verified.

Experiment-3: Expanding and Squeezing a NumPy Array

Perform the following operations using NumPy:

i) Expanding a NumPy Array

ii) Squeezing a NumPy Array

iii) Sorting in NumPy Arrays

AIM

To understand how to expand and squeeze the dimensions of NumPy arrays and perform sorting operations on one-dimensional and multidimensional arrays.

Description

In NumPy, the dimensions of an array can be modified without changing its data. Expanding an array increases its dimensions by inserting a new axis, while squeezing removes dimensions whose size is one. Sorting arranges array elements in ascending order by default.

These operations are commonly used in data preprocessing, machine learning, image processing, and scientific computing.

(i) Expanding a NumPy Array

The **expand_dims()** function inserts a new axis into an existing array.

This operation converts a lower-dimensional array into a higher-dimensional array.

For example,

- 1-D → 2-D
- 2-D → 3-D

Syntax

```
numpy.expand_dims(array, axis)
```

Where,

- **array** → Input array
- **axis** → Position where the new dimension is inserted

Example 1

```
import numpy as np

arr=np.array([10,20,30,40])

print(np.expand_dims(arr,axis=0))
```

Output

[[10 20 30 40]]

Example 2

```
import numpy as np

arr=np.array([10,20,30,40])

print(np.expand_dims(arr,axis=1))
```

Output

[[10]
 [20]
 [30]
 [40]]

(ii) Squeezing a NumPy Array

The **squeeze()** function removes dimensions whose size is equal to 1.

It converts higher-dimensional arrays into lower-dimensional arrays.

Syntax

```
numpy.squeeze(array)
```

Example

```
import numpy as np

arr=np.array([[[10,20,30]]])

print(arr.shape)

new_arr=np.squeeze(arr)

print(new_arr)

print(new_arr.shape)
```

Output

(1, 1, 3)
[10 20 30]
(3,)

(iii) Sorting in NumPy Arrays

The **sort()** function arranges elements in ascending order.

It can sort

- One-dimensional arrays
- Two-dimensional arrays
- Three-dimensional arrays

Syntax

```
numpy.sort(array)
```

Example 1 (1-D Array)

```
import numpy as np

arr=np.array([45,12,89,22,67])

print(np.sort(arr))
```

Output

[12 22 45 67 89]

Example 2 (2-D Array)

```
import numpy as np

arr=np.array([[45,12,89],
              [30,21,15]])

print(np.sort(arr))
```

Output

[[12 45 89]
 [15 21 30]]

Procedure

Step 1: Import the NumPy library.

import numpy as np

Step 2: Create a NumPy array.

Step 3: Expand the dimensions using expand_dims().

Step 4: Display the expanded array.

Step 5: Create an array with dimensions of size one.

Step 6: Remove unnecessary dimensions using squeeze().

Step 7: Create unsorted arrays.

Step 8: Sort the arrays using sort().

Step 9: Execute the program and observe the output.

Program

```
import numpy as np

arr=np.array([10,20,30,40])

print("Original Array")
print(arr)

print("\nExpanded Array (Axis=0)")
print(np.expand_dims(arr,axis=0))

print("\nExpanded Array (Axis=1)")
print(np.expand_dims(arr,axis=1))

arr2=np.array([[[10,20,30]]])

print("\nOriginal Shape")
print(arr2.shape)

sq=np.squeeze(arr2)

print("\nSqueezed Array")
print(sq)

print("\nShape after Squeeze")
print(sq.shape)


arr3=np.array([45,12,89,22,67])

print("\nSorted 1-D Array")
print(np.sort(arr3))

arr4=np.array([[45,12,89],
               [30,21,15]])

print("\nSorted 2-D Array")
print(np.sort(arr4))
```

Output

```
Original Array
[10 20 30 40]

Expanded Array (Axis=0)
[[10 20 30 40]]

Expanded Array (Axis=1)
[[10]
 [20]
 [30]
 [40]]

Original Shape
(1, 1, 3)

Squeezed Array
[10 20 30]

Shape after Squeeze
(3,)

Sorted 1-D Array
[12 22 45 67 89]

Sorted 2-D Array
[[12 45 89]
 [15 21 30]]
```

Result

Thus, the dimensions of NumPy arrays were successfully expanded and squeezed, and sorting operations on one-dimensional and two-dimensional arrays were performed successfully.

Experiment-4: Indexing and Slicing of NumPy Arrays

Perform the following operations using NumPy:

i) Slicing 1-D NumPy Arrays

ii) Slicing 2-D NumPy Arrays

iii) Slicing 3-D NumPy Arrays

iv) Negative Slicing of NumPy Arrays

AIM

To understand indexing and slicing techniques in NumPy arrays and access elements from one-dimensional, two-dimensional, and three-dimensional arrays.

Description

Indexing and slicing are fundamental operations in NumPy that allow us to access specific elements or groups of elements from an array.

Indexing is used to access individual elements.

Slicing is used to access multiple elements using a specified range.

The general syntax of slicing is:

```
array[start:stop:step]
```

Where,

- **start** – Starting index (included)
- **stop** – Ending index (excluded)
- **step** – Interval between elements

If any parameter is omitted, NumPy uses its default value.

(i) Slicing 1-D NumPy Arrays

A one-dimensional array contains elements arranged in a single row.

Example

```
import numpy as np

arr=np.array([10,20,30,40,50,60])

print(arr[1:5])
```

Output

[20 30 40 50]

Accessing Individual Elements

```
import numpy as np

arr=np.array([10,20,30,40,50])

print(arr[2])
```

Output

30

(ii) Slicing 2-D NumPy Arrays

A two-dimensional array contains rows and columns.

The syntax is

```
array[row_start:row_end,column_start:column_end]
```

Example

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60],
              [70,80,90]])

print(arr[0:2,1:3])
```

Output

[[20 30]
 [50 60]]

Accessing a Complete Row

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60],
              [70,80,90]])

print(arr[1])
```

Output

[40 50 60]

Accessing a Complete Column

```
import numpy as np

arr=np.array([[10,20,30],
              [40,50,60],
              [70,80,90]])

print(arr[:,2])
```

Output

[30 60 90]

(iii) Slicing 3-D NumPy Arrays

A three-dimensional array consists of multiple two-dimensional arrays.

Example

```
import numpy as np

arr=np.array([
[[1,2,3],
 [4,5,6]],

[[7,8,9],
 [10,11,12]]
])

print(arr[1,:,:])
```

Output

[[7 8 9]
 [10 11 12]]

Selecting Specific Elements

```
import numpy as np

arr=np.array([
[[1,2,3],
 [4,5,6]],

[[7,8,9],
 [10,11,12]]
])

print(arr[:,0,1])
```

Output

[2 8]

(iv) Negative Slicing of NumPy Arrays

Negative indexing starts from the end of the array.

The last element has index **-1**.

Example

```
import numpy as np

arr=np.array([10,20,30,40,50,60])

print(arr[-4:-1])
```

Output

[30 40 50]

Reverse an Array

```
import numpy as np

arr=np.array([10,20,30,40,50])

print(arr[::-1])
```

Output

[50 40 30 20 10]

Procedure

Step 1: Import the NumPy library.

import numpy as np

Step 2: Create a one-dimensional array.

Step 3: Perform indexing and slicing operations.

Step 4: Create a two-dimensional array.

Step 5: Access rows, columns and subarrays.

Step 6: Create a three-dimensional array.

Step 7: Perform slicing on different dimensions.

Step 8: Perform negative indexing and reverse the array.

Step 9: Execute the program and observe the output.

Program

```
import numpy as np

print("1-D Array")

a=np.array([10,20,30,40,50,60])

print(a)

print("\n1-D Slicing")
print(a[1:5])

print("\nElement at Index 2")
print(a[2])

print("\n2-D Array")

b=np.array([[10,20,30],
            [40,50,60],
            [70,80,90]])

print(b)

print("\nRows 1 to 2 and Columns 2 to 3")
print(b[0:2,1:3])
print("\nSecond Row")
print(b[1])

print("\nThird Column")
print(b[:,2])

print("\n3-D Array")

c=np.array([
[[1,2,3],
 [4,5,6]],

[[7,8,9],
 [10,11,12]]
])

print(c)
```

```
print("\nSecond Block")
print(c[1,:,:])

print("\nSelected Elements")
print(c[:,0,1])

print("\nNegative Slicing")
print(a[-4:-1])

print("\nReverse Array")
print(a[::-1])
```

Output

```
1-D Array
[10 20 30 40 50 60]

1-D Slicing
[20 30 40 50]

Element at Index 2
30

2-D Array
[[10 20 30]
 [40 50 60]
 [70 80 90]]

Rows 1 to 2 and Columns 2 to 3
[[20 30]
 [50 60]]

Second Row
[40 50 60]

Third Column
[30 60 90]

3-D Array
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
```

```
Second Block
[[ 7  8  9]
 [10 11 12]]

Selected Elements
[2 8]

Negative Slicing
[30 40 50]

Reverse Array
[60 50 40 30 20 10]
```

Result

Thus, indexing and slicing operations on one-dimensional, two-dimensional, and three-dimensional NumPy arrays were successfully performed. Negative indexing and reverse slicing were also verified.

Experiment-5: Stacking and Concatenating NumPy Arrays

Perform the following operations using NumPy:

i) Stacking ndarrays

ii) Concatenating ndarrays

iii) Broadcasting in NumPy Arrays

AIM

To understand how to combine multiple NumPy arrays using stacking and concatenation techniques and perform arithmetic operations using broadcasting.

Description

NumPy provides several functions to combine two or more arrays into a single array. Depending on the requirement, arrays can be joined vertically, horizontally, or along a specified axis.

Broadcasting is a powerful feature of NumPy that allows arithmetic operations between arrays of different shapes, provided they satisfy broadcasting rules.

These operations are widely used in data analysis, machine learning, image processing, and scientific computing.

(i) Stacking ndarrays

Stacking combines two or more arrays along a new axis.

NumPy provides the following stacking functions:

- vstack() – Vertical Stacking
- hstack() – Horizontal Stacking
- dstack() – Depth Stacking
- stack() – General Stacking

Vertical Stacking

The vstack() function stacks arrays vertically (row-wise).

Syntax

```
numpy.vstack((array1,array2))
```

Example

```
import numpy as np

a=np.array([1,2,3])

b=np.array([4,5,6])

print(np.vstack((a,b)))
```

Output

```
[[1 2 3]
 [4 5 6]]
```

Horizontal Stacking

The hstack() function stacks arrays horizontally (column-wise).

Syntax

```
numpy.hstack((array1,array2))
```

Example

```
import numpy as np

a=np.array([1,2,3])

b=np.array([4,5,6])

print(np.hstack((a,b)))
```

Output

[1 2 3 4 5 6]

Depth Stacking

The dstack() function stacks arrays along the third dimension.

Example

```
import numpy as np

a=np.array([[1,2],
            [3,4]])

b=np.array([[5,6],
            [7,8]])

print(np.dstack((a,b)))
```

Output

```
[[[1 5]
  [2 6]]
 [[3 7]
  [4 8]]]
```

(ii) Concatenating ndarrays

The concatenate() function joins two or more arrays along an existing axis.

Syntax

```
numpy.concatenate((array1,array2),axis=0)
```

Where,

- axis=0 → Row-wise concatenation

- axis=1 → Column-wise concatenation

Example

```
import numpy as np

a=np.array([[1,2],
            [3,4]])

b=np.array([[5,6],
            [7,8]])

print(np.concatenate((a,b),axis=0))
```

Output

[[1 2]

[3 4]

[5 6]

[7 8]]

Concatenation along Columns

```
import numpy as np

a=np.array([[1,2],
            [3,4]])

b=np.array([[5,6],
            [7,8]])

print(np.concatenate((a,b),axis=1))
```

Output

[[1 2 5 6]

[3 4 7 8]]

(iii) Broadcasting in NumPy Arrays

Broadcasting allows arithmetic operations between arrays having different shapes.

Instead of explicitly copying data, NumPy automatically expands the smaller array to match the larger array whenever possible.

Broadcasting is possible when

- Both arrays have the same dimensions, or
- One of the dimensions is equal to 1.

Example 1

```
import numpy as np

arr=np.array([10,20,30])

print(arr+5)
```

Output

[15 25 35]

Example 2

```
import numpy as np

a=np.array([[1],
            [2],
            [3]])

b=np.array([10,20,30])

print(a+b)
```

Output

[[11 21 31]
 [12 22 32]
 [13 23 33]]

Procedure

Step 1: Import the NumPy library.

import numpy as np

Step 2: Create two NumPy arrays.

Step 3: Perform vertical stacking using vstack().

Step 4: Perform horizontal stacking using hstack().

Step 5: Perform depth stacking using dstack().

Step 6: Join arrays using concatenate().

Step 7: Perform arithmetic operations using broadcasting.

Step 8: Execute the program and observe the output.

Program

```
import numpy as np

a=np.array([1,2,3])

b=np.array([4,5,6])

print("Vertical Stack")
print(np.vstack((a,b)))

print("\nHorizontal Stack")
print(np.hstack((a,b)))

c=np.array([[1,2],
            [3,4]])

d=np.array([[5,6],
            [7,8]])

print("\nDepth Stack")
print(np.dstack((c,d)))

print("\nConcatenate along Rows")
print(np.concatenate((c,d),axis=0))

print("\nConcatenate along Columns")
print(np.concatenate((c,d),axis=1))

print("\nBroadcasting Example")
arr=np.array([10,20,30])
print(arr+5)

print("\nBroadcasting between Arrays")
x=np.array([[1],
            [2],
            [3]])

y=np.array([10,20,30])

print(x+y)
```

Output

Vertical Stack

[[1 2 3]

[4 5 6]]

Horizontal Stack

[1 2 3 4 5 6]

Depth Stack

[[[1 5]
[2 6]]
[[3 7]
[4 8]]]
Concatenate along Rows
[[1 2]
[3 4]
[5 6]
[7 8]]
Concatenate along Columns
[[1 2 5 6]
[3 4 7 8]]
Broadcasting Example
[15 25 35]
Broadcasting between Arrays
[[11 21 31]
[12 22 32]
[13 23 33]]

Result

Thus, stacking, concatenation, and broadcasting operations on NumPy arrays were successfully performed and verified.

Experiment-6: Perform the Following Operations Using Pandas

Perform the following operations using Pandas:

i) Creating DataFrame

ii) concat()

iii) Setting Conditions

iv) Adding a New Column

AIM

To understand the basic operations of the Pandas library by creating DataFrames, concatenating DataFrames, filtering data using conditions, and adding new columns.

Description

Pandas is an open-source Python library used for data manipulation and analysis. It provides powerful data structures such as **Series** and **DataFrame** for handling structured data efficiently.

A **DataFrame** is a two-dimensional labeled data structure consisting of rows and columns. It is one of the most commonly used data structures in data analysis.

Pandas provides various functions to create, manipulate, merge, filter, and analyze data.

(i) Creating DataFrame

A DataFrame can be created using dictionaries, lists, tuples, NumPy arrays, or external files.

Syntax

```
pandas.DataFrame(data)
```

Example

```
import pandas as pd

data={
    "RollNo":[101,102,103],
    "Name":["Rahul","Priya","Kiran"],
    "Marks":[85,90,78]
}

df=pd.DataFrame(data)

print(df)
```

Output

```
    RollNo    Name   Marks
0      101   Rahul      85
1      102   Priya      90
2      103   Kiran      78
```

(ii) concat()

The **concat()** function combines two or more DataFrames.

Syntax

```
pandas.concat([df1,df2])
```

Example

```
import pandas as pd

df1=pd.DataFrame({
    "Name":["A","B"],
    "Marks":[80,85]
})

df2=pd.DataFrame({
    "Name":["C","D"],
    "Marks":[75,90]
})

print(pd.concat([df1,df2]))
```

Output

```
  Name  Marks
0    A     80
1    B     85
0    C     75
1    D     90
```

(iii) Setting Conditions

Conditions are used to filter rows based on specified criteria.

Syntax

```
dataframe[condition]
```

Example

```
import pandas as pd

data={
    "Name":["A","B","C","D"],
    "Marks":[70,85,90,60]
}

df=pd.DataFrame(data)

print(df[df["Marks"]>=80])
```

Output

```
  Name  Marks
1    B     85
2    C     90
```

(iv) Adding a New Column

A new column can be added by assigning values to a new column name.

Syntax

```
dataframe["ColumnName"]=values
```

Example

```
import pandas as pd

data={
    "Name":["A","B","C"],
    "Marks":[70,80,90]
}

df=pd.DataFrame(data)

df["Result"]=["Pass","Pass","Pass"]

print(df)
```

Output

```
  Name  Marks Result
0    A     70   Pass
1    B     80   Pass
2    C     90   Pass
```

Procedure

Step 1: Install the Pandas library.

pip install pandas

Step 2: Import the Pandas library.

import pandas as pd

Step 3: Create a DataFrame.

Step 4: Create another DataFrame and combine both using concat().

Step 5: Apply conditions to filter rows.

Step 6: Add a new column to the DataFrame.

Step 7: Execute the program.

Step 8: Observe the output.

Program

```
import pandas as pd

student1={
    "RollNo":[101,102,103],
    "Name":["Rahul","Priya","Kiran"],
    "Marks":[85,72,91]
}

student2={
    "RollNo":[104,105],
    "Name":["Anil","Sneha"],
    "Marks":[76,88]
}

df1=pd.DataFrame(student1)
df2=pd.DataFrame(student2)

print("First DataFrame")
print(df1)

print("\nSecond DataFrame")
print(df2)

print("\nConcatenated DataFrame")
df=pd.concat([df1,df2])
print(df)

print("\nStudents Scoring 80 or More Marks")
print(df[df["Marks"]>=80])

df["Result"]="Pass"

print("\nDataFrame After Adding New Column")
print(df)
```

Output

```
First DataFrame

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     72
2     103  Kiran     91

Second DataFrame

   RollNo   Name  Marks
0     104   Anil     76
1     105  Sneha     88

Concatenated DataFrame

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     72
2     103  Kiran     91
0     104   Anil     76
1     105  Sneha     88

Students Scoring 80 or More Marks

   RollNo   Name  Marks
0     101  Rahul     85
2     103  Kiran     91
1     105  Sneha     88

DataFrame After Adding New Column

   RollNo   Name  Marks Result
0     101  Rahul     85   Pass
1     102  Priya     72   Pass
2     103  Kiran     91   Pass
0     104   Anil     76   Pass
1     105  Sneha     88   Pass
```

Result

Thus, the basic operations using Pandas such as creating a DataFrame, concatenating DataFrames, filtering data using conditions, and adding a new column were successfully performed.

Experiment-7: Perform the Following Operations Using Pandas

Perform the following operations using Pandas:

i) Filling NaN with String

ii) Sorting Based on Column Values

iii) groupby()

AIM

To understand how to handle missing values, sort data based on column values, and group similar data using the Pandas library.

Description

In real-world datasets, missing values, unordered records, and repeated categories are common. Pandas provides built-in functions to clean and organize such data efficiently.

Missing values are represented as **NaN (Not a Number)**. These values can be replaced using the **fillna()** function.

The **sort_values()** function arranges the records in ascending or descending order.

The **groupby()** function groups rows having common values and performs aggregate operations such as sum, mean, count, maximum, and minimum.

These operations are widely used during data preprocessing and exploratory data analysis.

(i) Filling NaN with String

The **fillna()** function replaces missing values with a specified value.

Syntax

```
dataframe.fillna(value)
```

Example

```
import pandas as pd
import numpy as np

data={
    "Name":["Rahul","Priya",np.nan,"Anil"],
    "City":["Hyderabad",np.nan,"Chennai","Delhi"]
}

df=pd.DataFrame(data)

print(df.fillna("Not Available"))
```

Output

```
              Name            City
0            Rahul       Hyderabad
1            Priya   Not Available
2   Not Available         Chennai
3             Anil           Delhi
```

(ii) Sorting Based on Column Values

The **sort_values()** function arranges the DataFrame according to one or more columns.

Syntax

```
dataframe.sort_values(by="ColumnName")
```

Example

```
import pandas as pd

data={
    "Name":["Rahul","Priya","Anil","Sneha"],
    "Marks":[75,92,81,65]
}

df=pd.DataFrame(data)

print(df.sort_values(by="Marks"))
```

Output

```
     Name  Marks
3   Sneha     65
0   Rahul     75
2    Anil     81
1   Priya     92
```

Sorting in Descending Order

Example

```
import pandas as pd

data={
    "Name":["Rahul","Priya","Anil"],
    "Marks":[75,92,81]
}

df=pd.DataFrame(data)

print(df.sort_values(by="Marks",ascending=False))
```

Output

```
    Name  Marks
1  Priya     92
2   Anil     81
0  Rahul     75
```

(iii) groupby()

The **groupby()** function groups rows having the same value in a specified column.

After grouping, aggregate functions such as **sum(), mean(), max(), min(), count()** can be applied.

Syntax

```
dataframe.groupby("ColumnName")
```

Example

```
import pandas as pd

data={
    "Department":["CSE","IT","CSE","IT","AIML"],
    "Marks":[80,75,90,85,95]
}

df=pd.DataFrame(data)

print(df.groupby("Department")["Marks"].mean())
```

Output

```
Department
AIML    95.0
CSE     85.0
IT      80.0
Name: Marks, dtype: float64
```

Another Example

```
import pandas as pd

data={
    "Department":["CSE","IT","CSE","IT","AIML"],
    "Marks":[80,75,90,85,95]
}

df=pd.DataFrame(data)

print(df.groupby("Department")["Marks"].sum())
```

Output

```
Department
AIML     95
CSE     170
IT      160
Name: Marks, dtype: int64
```

Procedure

Step 1: Import the Pandas and NumPy libraries.

import pandas as pd

import numpy as np

Step 2: Create a DataFrame containing missing values.

Step 3: Replace the missing values using fillna().

Step 4: Create another DataFrame containing student records.

Step 5: Sort the DataFrame using sort_values().

Step 6: Group the records using groupby().

Step 7: Display the average marks of each department.

Step 8: Execute the program and observe the output.

Program

```
import pandas as pd
import numpy as np

data={
    "Name":["Rahul","Priya",np.nan,"Anil"],
    "Department":["CSE","IT","CSE","AIML"],
    "Marks":[75,92,81,88]
}

df=pd.DataFrame(data)

print("Original DataFrame")
print(df)

print("\nAfter Filling NaN")
print(df.fillna("Not Available"))

print("\nSorted by Marks")
print(df.sort_values(by="Marks"))

print("\nSorted by Marks (Descending)")
print(df.sort_values(by="Marks",ascending=False))

print("\nAverage Marks Department Wise")
print(df.groupby("Department")["Marks"].mean())
```

Output

```
Original DataFrame

    Name Department  Marks
0  Rahul        CSE     75
1  Priya         IT     92
2    NaN        CSE     81
3   Anil       AIML     88

After Filling NaN

             Name Department  Marks
0           Rahul        CSE     75
1           Priya         IT     92
2  Not Available        CSE     81
3            Anil       AIML     88

Sorted by Marks

    Name Department  Marks
0  Rahul        CSE     75
2    NaN        CSE     81
3   Anil       AIML     88
1  Priya         IT     92

Sorted by Marks (Descending)

    Name Department  Marks
1  Priya         IT     92
3   Anil       AIML     88
2    NaN        CSE     81
0  Rahul        CSE     75

Average Marks Department Wise

Department
AIML    88.0
CSE     78.0
IT      92.0
Name: Marks, dtype: float64
```

Result

Thus, the operations of filling missing values using **fillna()**, sorting records using **sort_values()**, and grouping data using **groupby()** were successfully performed using the Pandas library.

Experiment-8: Read the Following File Formats Using Pandas

Perform the following operations using Pandas:

i) Read Text Files

ii) Read CSV Files

iii) Read Excel Files

iv) Read JSON Files

AIM

To understand how to import data from different file formats such as Text, CSV, Excel, and JSON files using the Pandas library.

Description

Data is stored in different file formats depending on the application. Pandas provides built-in functions to read these files and convert them into DataFrames for analysis.

The commonly used file formats are:

- Text (.txt)
- CSV (.csv)
- Excel (.xlsx)
- JSON (.json)

After importing the data, the DataFrame can be used for filtering, preprocessing, visualization, and analysis.

(i) Reading Text Files

A text file can be read using the **read_csv()** function by specifying the delimiter.

Syntax

```
pandas.read_csv("filename.txt", delimiter=" ")
```

Example

Suppose the file **student.txt** contains

101 Rahul 85

102 Priya 90

103 Anil 78

Program

```
import pandas as pd

df=pd.read_csv("student.txt",delimiter=" ",
               names=["RollNo","Name","Marks"])

print(df)
```

Output

```
   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

(ii) Reading CSV Files

CSV (Comma Separated Values) is one of the most commonly used file formats for storing tabular data.

The **read_csv()** function is used to read CSV files.

Syntax

```
pandas.read_csv("filename.csv")
```

Example

Suppose the file **student.csv** contains

RollNo,Name,Marks

101,Rahul,85

102,Priya,90

103,Anil,78

Program

```
import pandas as pd

df=pd.read_csv("student.csv")

print(df)
```

Output

```
   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

(iii) Reading Excel Files

Pandas provides the **read_excel()** function to import Excel files.

Syntax

```
pandas.read_excel("filename.xlsx")
```

Example

```
import pandas as pd

df=pd.read_excel("student.xlsx")

print(df)
```

Output

```
   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

(iv) Reading JSON Files

JSON (JavaScript Object Notation) is widely used for exchanging data between applications.

The **read_json()** function imports JSON data into a DataFrame.

Syntax

```
pandas.read_json("filename.json")
```

Example

Suppose **student.json** contains

```
[
 {"RollNo":101,"Name":"Rahul","Marks":85},
 {"RollNo":102,"Name":"Priya","Marks":90},
 {"RollNo":103,"Name":"Anil","Marks":78}
]
```

Program

```
import pandas as pd

df=pd.read_json("student.json")

print(df)
```

Output

```
   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

Procedure

Step 1: Import the Pandas library.

import pandas as pd

Step 2: Create the required input files (**student.txt**, **student.csv**, **student.xlsx**, and **student.json**).

Step 3: Read the text file using read_csv().

Step 4: Read the CSV file using read_csv().

Step 5: Read the Excel file using read_excel().

Step 6: Read the JSON file using read_json().

Step 7: Display the imported DataFrames.

Step 8: Execute the program and observe the output.

Program

```
import pandas as pd

print("Reading Text File")
text_df=pd.read_csv("student.txt",
                    delimiter=" ",
                    names=["RollNo","Name","Marks"])
print(text_df)

print("\nReading CSV File")
csv_df=pd.read_csv("student.csv")
print(csv_df)

print("\nReading Excel File")
excel_df=pd.read_excel("student.xlsx")
print(excel_df)

print("\nReading JSON File")
json_df=pd.read_json("student.json")
print(json_df)
```

C

Output

```
Reading Text File

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78

Reading CSV File

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

```
Reading Excel File

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78

Reading JSON File

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

Result

Thus, different file formats such as **Text**, **CSV**, **Excel**, and **JSON** files were successfully imported into Pandas DataFrames.

Experiment-9: Read the Following File Formats

Perform the following operations:

i) Read Pickle Files

ii) Read Image Files using PIL

iii) Read Multiple Files using Glob

iv) Import Data from Database

AIM

To understand how to read Pickle files, image files, multiple files, and data from a database using Python libraries.

Description

Python supports reading data from various file formats and external data sources. Besides text-based files such as CSV and Excel, data can also be stored as serialized objects, images, multiple files in a directory, and relational databases.

The following libraries are used in this experiment:

- **Pandas** – Reading Pickle files
- **PIL (Python Imaging Library)** – Reading image files
- **Glob** – Reading multiple files using wildcard patterns
- **SQLite3** – Importing data from a database

These libraries are widely used in data analysis and data preprocessing.

(i) Reading Pickle Files

A Pickle file stores Python objects in a binary format.

The **read_pickle()** function reads Pickle files into a DataFrame.

Syntax

```
pandas.read_pickle("filename.pkl")
```

Example

```
import pandas as pd

df=pd.read_pickle("student.pkl")

print(df)
```

Output

```
   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

(ii) Reading Image Files using PIL

The Python Imaging Library (PIL) is used to open and process image files.

The **Image.open()** function opens an image.

Syntax

```
Image.open("filename")
```

Example

```
from PIL import Image

img=Image.open("flower.jpg")

print(img)

img.show()
```

Output

```
<PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=800x600>
```

An image window displaying **flower.jpg** will be opened.

(iii) Reading Multiple Files using Glob

The **glob** module is used to read multiple files matching a specified pattern.

Syntax

```
glob.glob("pattern")
```

Example

```
import glob

files=glob.glob("*.csv")

print(files)
```

Output

```
['student.csv', 'marks.csv', 'employee.csv']
```

Reading all CSV files

```
import glob
import pandas as pd

files=glob.glob("*.csv")

for file in files:
    df=pd.read_csv(file)
    print(file)
    print(df)
```

Output

```
student.csv
<contents of student.csv>

marks.csv
<contents of marks.csv>

employee.csv
<contents of employee.csv>
```

(iv) Importing Data from Database

Python provides the **sqlite3** library for connecting to SQLite databases.

The **read_sql_query()** function reads records from a database table into a DataFrame.

Syntax

```
pandas.read_sql_query(query,connection)
```

Example

```
import sqlite3
import pandas as pd

conn=sqlite3.connect("student.db")

df=pd.read_sql_query("SELECT * FROM Student",conn)

print(df)

conn.close()
```

Output

```
    RollNo    Name  Marks
0      101   Rahul     85
1      102   Priya     90
2      103    Anil     78
```

Procedure

Step 1: Install the required libraries.

```
pip install pandas pillow
```

Step 2: Import the required libraries.

```
import pandas as pd
from PIL import Image
import glob
import sqlite3
```

Step 3: Read the Pickle file using read_pickle().

Step 4: Open an image using Image.open().

Step 5: Read multiple CSV files using glob.glob().

Step 6: Connect to the SQLite database.

Step 7: Read records from the database using read_sql_query().

Step 8: Display the outputs.

Program

```
import pandas as pd
from PIL import Image
import glob
import sqlite3

print("Reading Pickle File")

pickle_df=pd.read_pickle("student.pkl")
print(pickle_df)

print("\nReading Image File")

img=Image.open("flower.jpg")
print(img)

print("\nReading Multiple CSV Files")

files=glob.glob("*.csv")

for file in files:
    print(file)

print("\nReading Data from Database")

conn=sqlite3.connect("student.db")

df=pd.read_sql_query("SELECT * FROM Student",conn)

print(df)

conn.close()
```

Output

```
Reading Pickle File

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78

Reading Image File

<PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=800x600>

Reading Multiple CSV Files

student.csv
marks.csv
employee.csv

Reading Data from Database

   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Anil     78
```

Result

Thus, Pickle files, image files, multiple files using the Glob module, and data from an SQLite database were successfully read using Python.

Experiment-10: Demonstrate Web Scraping Using Python

AIM

To understand the basics of web scraping using Python and extract information from a web page using the Requests and BeautifulSoup libraries.

Description

Web Scraping is the process of automatically collecting data from websites. Instead of manually copying information from web pages, Python programs can retrieve the HTML source code of a webpage and extract the required information.

Python provides several libraries for web scraping. The most commonly used libraries are:

- **Requests** – Used to send HTTP requests to web servers and retrieve web page content.
- **BeautifulSoup** – Used to parse HTML or XML documents and extract useful information.

The general steps involved in web scraping are:

1. Send a request to a website.
2. Receive the HTML page.
3. Parse the HTML content.
4. Locate the required HTML elements.
5. Extract and display the required information.

(i) Sending a Request to a Website

The **requests** library is used to retrieve the HTML content of a webpage.

Syntax

```
requests.get(url)
```

Example

```
import requests

url="https://example.com"

response=requests.get(url)

print(response.status_code)
```

Output

200

The status code **200** indicates that the webpage was successfully retrieved.

(ii) Parsing HTML using BeautifulSoup

BeautifulSoup converts HTML into a parse tree that can be searched easily.

Syntax

```
BeautifulSoup(html_content,"html.parser")
```

Example

```
import requests
from bs4 import BeautifulSoup

url="https://example.com"

response=requests.get(url)

soup=BeautifulSoup(response.text,"html.parser")

print(soup.title.text)
```

Output

Example Domain

(iii) Extracting Headings

The **find()** method retrieves the first matching HTML element.

Syntax

```
soup.find("tagname")
```

Example

```
import requests
from bs4 import BeautifulSoup

url="https://example.com"

response=requests.get(url)

soup=BeautifulSoup(response.text,"html.parser")

heading=soup.find("h1")

print(heading.text)
```

Output

Example Domain

(iv) Extracting Multiple Elements

The **find_all()** method retrieves all matching HTML elements.

Syntax

```
soup.find_all("tagname")
```

Example

```
import requests
from bs4 import BeautifulSoup

url="https://books.toscrape.com/"

response=requests.get(url)

soup=BeautifulSoup(response.text,"html.parser")

titles=soup.find_all("h3")

for title in titles:
    print(title.text)
```

Sample Output

A Light in the Attic

Tipping the Velvet

Soumission

Sharp Objects

...

Procedure

Step 1: Install the required libraries.

```
pip install requests
pip install beautifulsoup4
```

Step 2: Import the required libraries.

```
import requests
from bs4 import BeautifulSoup
```

Step 3: Specify the URL of the webpage.

Step 4: Send an HTTP request using requests.get().

Step 5: Parse the HTML page using BeautifulSoup.

Step 6: Extract the webpage title.

Step 7: Extract the heading from the webpage.

Step 8: Display the extracted information.

Program

```
import requests
from bs4 import BeautifulSoup

url="https://example.com"

response=requests.get(url)

soup=BeautifulSoup(response.text,"html.parser")

print("Status Code")
print(response.status_code)

print("\nPage Title")
print(soup.title.text)

print("\nHeading")
print(soup.find("h1").text)

print("\nParagraph")

print(soup.find("p").text)
```

Output

```
Status Code
200

Page Title
Example Domain

Heading
Example Domain

Paragraph
This domain is for use in illustrative examples in documents.
```

Result

Thus, web scraping was successfully performed using Python by retrieving a webpage, parsing its HTML content using BeautifulSoup, and extracting the required information.

Experiment-11: Perform the Following Preprocessing Techniques on Loan Prediction Dataset

Perform the following preprocessing techniques:

i) Feature Scaling

ii) Feature Standardization

iii) Label Encoding

iv) One Hot Encoding

AIM

To understand various data preprocessing techniques such as Feature Scaling, Standardization, Label Encoding, and One Hot Encoding on the Loan Prediction dataset using Python.

Description

Data preprocessing is one of the most important steps in data analysis and machine learning. Real-world datasets often contain attributes with different scales and categorical values. These datasets must be transformed into a suitable format before analysis or model building.

Common preprocessing techniques include:

- Feature Scaling
- Feature Standardization
- Label Encoding
- One Hot Encoding

These techniques improve data quality and help machine learning algorithms perform efficiently.

(i) Feature Scaling

Feature Scaling transforms numerical features into a fixed range.

The **MinMaxScaler** scales the data between **0 and 1**.

Formula

Syntax

```
from sklearn.preprocessing import MinMaxScaler

scaler=MinMaxScaler()

scaled_data=scaler.fit_transform(data)
```

Example

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data=pd.DataFrame({
    "Income":[25000,40000,55000,70000]
})

scaler=MinMaxScaler()

print(scaler.fit_transform(data))
```

Output

```
[[0.        ]
 [0.33333333]
 [0.66666667]
 [1.        ]]
```

(ii) Feature Standardization

Feature Standardization transforms data so that it has:

- Mean = 0
- Standard Deviation = 1

It is performed using **StandardScaler**.

Syntax

```
from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()

scaled=scaler.fit_transform(data)
```

Example

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

data=pd.DataFrame({
    "Income":[25000,40000,55000,70000]
})

scaler=StandardScaler()

print(scaler.fit_transform(data))
```

Sample Output

[[-1.3416]

[-0.4472]

[0.4472]

[1.3416]]

(iii) Label Encoding

Label Encoding converts categorical values into numerical values.

Example

Gender
Male
Female
Male

becomes

Gender
1
0
1

Syntax

```
from sklearn.preprocessing import LabelEncoder

encoder=LabelEncoder()

data["Gender"]=encoder.fit_transform(data["Gender"])
```

Example

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

data=pd.DataFrame({
    "Gender":["Male","Female","Male","Female"]
})

encoder=LabelEncoder()

data["Gender"]=encoder.fit_transform(data["Gender"])

print(data)
```

Output

```
   Gender
0       1
1       0
2       1
3       0
```

(iv) One Hot Encoding

One Hot Encoding converts each category into separate binary columns.

Example

City
Hyderabad
Chennai
Delhi

becomes

Hyderabad	Chennai	Delhi
1	0	0
0	1	0
0	0	1

Syntax

```
pandas.get_dummies(data,column_name)
```

Example

```
import pandas as pd

data=pd.DataFrame({
    "City":["Hyderabad","Delhi","Chennai"]
})

print(pd.get_dummies(data))
```

Output

```
   City_Chennai  City_Delhi  City_Hyderabad
0         False       False            True
1         False        True           False
2          True       False           False
```

Procedure

Step 1: Install the required libraries.

```
pip install pandas
pip install scikit-learn
```

Step 2: Import the required libraries.

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
```

Step 3: Load the Loan Prediction dataset.

Step 4: Apply Feature Scaling using MinMaxScaler.

Step 5: Apply Feature Standardization using StandardScaler.

Step 6: Apply Label Encoding on categorical columns.

Step 7: Apply One Hot Encoding using get_dummies().

Step 8: Display the transformed dataset.

Program

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder

df=pd.read_csv("loan_prediction.csv")

print("Original Dataset")

print(df.head())

scaler=MinMaxScaler()

df["ApplicantIncome"]=scaler.fit_transform(df[["ApplicantIncome"]])

standard=StandardScaler()

df["LoanAmount"]=standard.fit_transform(df[["LoanAmount"]])

 encoder=LabelEncoder()

 df["Gender"]=encoder.fit_transform(df["Gender"])

 df=pd.get_dummies(df,columns=["Property_Area"])

 print("\nPreprocessed Dataset")

 print(df.head())
```

s

Sample Output

Original Dataset

	Gender	Married	ApplicantIncome	LoanAmount	Property_Area
0	Male	No	5849	128	Urban
1	Male	Yes	4583	128	Rural
2	Male	Yes	3000	66	Urban
3	Male	Yes	2583	120	Semiurban
4	Male	No	6000	141	Urban

Preprocessed Dataset

	Gender	Married	ApplicantIncome	LoanAmount	Property_Area_Rural
0	1	No	0.0705	0.125	False
1	1	Yes	0.0548	0.125	True
2	1	Yes	0.0352	-0.842	False
3	1	Yes	0.0301	0.001	False
4	1	No	0.0724	0.328	False

Result

Thus, Feature Scaling, Feature Standardization, Label Encoding, and One Hot Encoding were successfully performed on the Loan Prediction dataset.

Experiment-12: Perform the Following Visualizations Using Matplotlib

Perform the following visualizations using Matplotlib:

i) Bar Graph

ii) Pie Chart

iii) Box Plot

iv) Histogram

v) Line Chart and Subplots

vi) Scatter Plot

AIM

To understand the basics of data visualization using the Matplotlib library and create different types of charts for representing data graphically.

Description

Data Visualization is the graphical representation of data. It helps users understand patterns, trends, relationships, and distributions more effectively than tables of numerical values.

Matplotlib is one of the most popular Python libraries used for creating static, interactive, and animated visualizations.

Some commonly used plots are:

- Bar Graph
- Pie Chart
- Box Plot
- Histogram
- Line Chart
- Scatter Plot

These visualizations are widely used in Data Analysis, Business Intelligence, Machine Learning, and Scientific Research.

(i) Bar Graph

A Bar Graph is used to compare values among different categories.

Syntax

```
matplotlib.pyplot.bar(x,y)
```

Example

```
import matplotlib.pyplot as plt

subjects=["Python","Java","C","C++"]

students=[60,45,35,50]

plt.bar(subjects,students)

plt.title("Students Enrolled")

plt.xlabel("Subjects")

plt.ylabel("Students")

plt.show()
```

Output

A bar graph displaying the number of students enrolled in different subjects.

(ii) Pie Chart

A Pie Chart represents data as portions of a circle.

Syntax

```
matplotlib.pyplot.pie(values,labels=labels)
```

Example

```
import matplotlib.pyplot as plt

languages=["Python","Java","C++","C"]

students=[45,25,20,10]

plt.pie(students,
        labels=languages,
        autopct="%1.1f%%")

plt.title("Programming Languages")

plt.show()
```

Output

A pie chart showing the percentage distribution of programming languages.

(iii) Box Plot

A Box Plot is used to represent the distribution of data and identify outliers.

Syntax

```
matplotlib.pyplot.boxplot(data)
```

Example

```
import matplotlib.pyplot as plt

marks=[65,70,72,75,80,82,85,90,95]

plt.boxplot(marks)

plt.title("Student Marks")

plt.show()
```

Output

A box plot showing the spread of marks.

(iv) Histogram

A Histogram shows the frequency distribution of numerical data.

Syntax

```
matplotlib.pyplot.hist(data)
```

Example

```
import matplotlib.pyplot as plt

marks=[65,70,72,75,80,82,85,90,95]

plt.hist(marks,bins=5)

plt.title("Histogram of Marks")

plt.xlabel("Marks")

plt.ylabel("Frequency")

plt.show()
```

Output

A histogram showing the frequency distribution of marks.

(v) Line Chart and Subplots

A Line Chart is used to show trends over a period of time.

Syntax

```
matplotlib.pyplot.plot(x,y)
```

Example

```
import matplotlib.pyplot as plt

days=[1,2,3,4,5]

sales=[120,150,180,170,210]

plt.plot(days,sales,marker="o")

plt.title("Daily Sales")

plt.xlabel("Day")

plt.ylabel("Sales")

plt.show()
```

Output

A line graph showing daily sales.

Subplots

Subplots allow multiple graphs to be displayed in the same figure.

Example

```
import matplotlib.pyplot as plt

x=[1,2,3,4]

y1=[2,4,6,8]

y2=[1,3,5,7]

plt.subplot(1,2,1)

plt.plot(x,y1)

plt.title("Graph 1")

plt.subplot(1,2,2)

plt.plot(x,y2)

plt.title("Graph 2")

plt.show()
```

Output

A figure displaying two line graphs side by side.

(vi) Scatter Plot

A Scatter Plot is used to display the relationship between two numerical variables.

Syntax

```
matplotlib.pyplot.scatter(x,y)
```

Example

```
import matplotlib.pyplot as plt

height=[150,155,160,165,170]

weight=[45,50,55,60,65]

plt.scatter(height,weight)

plt.title("Height vs Weight")

plt.xlabel("Height")

plt.ylabel("Weight")

plt.show()
```

Output

A scatter plot showing the relationship between height and weight.

Procedure

Step 1: Install the Matplotlib library.

```
pip install matplotlib
```

Step 2: Import the Matplotlib library.

```
import matplotlib.pyplot as plt
```

Step 3: Create sample datasets.

Step 4: Draw a Bar Graph.

Step 5: Draw a Pie Chart.

Step 6: Draw a Box Plot.

Step 7: Draw a Histogram.

Step 8: Draw a Line Chart and Subplots.

Step 9: Draw a Scatter Plot.

Step 10: Observe the generated graphs.

Program

```
import matplotlib.pyplot as plt

subjects=["Python","Java","C","C++"]
students=[60,45,35,50]

plt.bar(subjects,students)
plt.title("Bar Graph")
plt.show()

plt.pie(students,
        labels=subjects,
        autopct="%1.1f%%")
plt.title("Pie Chart")
plt.show()

marks=[65,70,72,75,80,82,85,90,95]

plt.boxplot(marks)
plt.title("Box Plot")
plt.show()

plt.hist(marks,bins=5)
plt.title("Histogram")
plt.show()


days=[1,2,3,4,5]
sales=[120,150,180,170,210]

plt.plot(days,sales,marker="o")
plt.title("Line Chart")
plt.show()

plt.subplot(1,2,1)
plt.plot([1,2,3],[2,4,6])

plt.title("Graph 1")

plt.subplot(1,2,2)
plt.plot([1,2,3],[1,3,5])
```

```
plt.title("Graph 2")

plt.show()

height=[150,155,160,165,170]
weight=[45,50,55,60,65]

plt.scatter(height,weight)

plt.title("Scatter Plot")

plt.show()
```

Output

1. Bar Graph displaying the number of students enrolled in different subjects.
2. Pie Chart showing the percentage distribution of students.
3. Box Plot representing the spread of marks.
4. Histogram showing the frequency distribution of marks.
5. Line Chart displaying the sales trend.
6. Subplots displaying two line graphs in a single figure.
7. Scatter Plot showing the relationship between height and weight.

Result

Thus, various data visualizations such as Bar Graph, Pie Chart, Box Plot, Histogram, Line Chart, Subplots, and Scatter Plot were successfully created using the Matplotlib library.